7.3 보안 감사 및 코드 리뷰 과정 분석

개요

Composable Finance 프로젝트는 다층적이고 체계적인 보안 감사 및 코드 리뷰 프로세스를 통해 높은 수준의 보안성과 코드 품질을 유지합니다. 이 과정은 외부 보안 감사, 내부 코드 리뷰, 자동화된 보안 검사, 그리고 의존성 관리를 포함하는 포괄적인 보안 전략으로 구성됩니다.

7.3.1 보안 감사 보고서 분석 (`audits/` 디렉토리)

감사 보고서 구조

프로젝트는 다양한 보안 감사 기관으로부터 정기적인 보안 감사를 받고 있습니다:

```
audits/
├── README.md // 감사 개요 및 참조
├── REFERENCES // 감사 참조 커밋
├── halborn/ // Halborn 보안 감사
├── picasso-cosmos/ // Picasso Cosmos 체인 감사
├── solana-restaking-v2/ // Solana 리스테이킹 감사
├── solana-bridge-contract/ // Solana 브릿지 계약 감사
├── solana-ibc-avs/ // Solana IBC AVS 감사
└── mantis-contracts/ // Mantis 계약 감사
```

주요 보안 감사 이력

Halborn 보안 감사 시리즈:

```
# 주요 Halborn 감사 보고서

- 메인 팰릿 감사 (2021.02.29): audit20210229-pallets-core.pdf

- Pablo V2 감사 (2022.07.30): audit20220730-pallets-pabloV2.pdf

- Democracy, Currency Factory & Crowdloan Rewards (2022.08.08)

- Vesting 팰릿 감사 (2022.08.23): audit20220823-pallet-vesting.pdf

- BYOG 팰릿 감사 (2022.09.26): audit20220926-pallet-byog.pdf

- Grandpa Light Client Bridge (2022.10.03)

- IBC 팰릿 감사 (2022.10.10): audit20221010-pallet-ibc.pdf

- Pablo 리팩토링 감사 (2022.11.29): audit20221129-pallet-pablo-refactoring.pdf
```

최신 감사 보고서:

```
# 최근 감사 활동

- Codezen Audit - Picasso Cosmos chain (2024.11.01)

- OtterSec Audit - Solana Restaking Vault (2024.02.14)
```

감사 범위 및 커버리지

감사는 다음과 같은 핵심 영역을 포함합니다:

1. **핵심 팰릿 보안**

- Pablo DEX 팰릿의 AMM 로직
- Liquid Staking 팰릿의 스테이킹 메커니즘
- Oracle 팰릿의 가격 피드 시스템

2. **크로스체인 보안**

- IBC 프로토콜 구현
- XCM 메시지 처리
- 브릿지 계약 보안

3. **거버넌스 및 권한 관리**

- Democracy 팰릿
- 권한 관리 시스템
- 업그레이드 메커니즘

7.3.2 코드 리뷰 프로세스

Pull Request 템플릿

프로젝트는 체계적인 PR 템플릿을 통해 일관된 코드 리뷰를 보장합니다:

```
# PR 체크리스트 (pull_request_template.md)

Required for merge:

- [ ] `pr-workflow-check / draft-release-check` is ✅ success


Makes review faster:

- [ ] PR title provides clear summary for release notes

- [ ] Marked with `misc` label if not for release notes

- [ ] Linked relevant references (Zenhub/Github/Slack)

- [ ] Clear deployment requirements specified

- [ ] Added appropriate reviewers
```

```
- [ ] Tagged specific reviewer for best review
- [ ] Proved no general regressions
- [ ] Dependency updates follow guidelines
```

리뷰어 가이드라인

[REVIEWERS.md](#)에 정의된 코드 리뷰 원칙:

```
# 코드 리뷰 원칙
1. 항상 main 브랜치에서 분기
2. 브랜치명에 개발자 이름 포함
3. 팀 작업시 feat/{feature} 브랜치 사용
4. 코드 리뷰 도구의 설명은 미래 독자에게 도움이 되어야 함
5. non-nit 코멘트는 새로운 커밋으로 수정되어야 함
```

리뷰 품질 보장

Google의 Engineering Practices 기반 리뷰 프로세스:

1. **코드 정확성**: 기능적 정확성 검증
2. **보안성**: 보안 취약점 검토
3. **성능**: 성능 영향 분석
4. **가독성**: 코드 가독성 및 유지보수성
5. **테스트**: 적절한 테스트 커버리지

7.3.3 자동화된 보안 검사

Cargo Deny 설정

deny.toml을 통한 의존성 보안 정책:

```
[advisories]
# 보안 취약점 정책
vulnerability = "deny"  # 보안 취약점 거부
unmaintained = "warn"  # 유지보수 중단 크레이트 경고
yanked = "warn"  # 철회된 크레이트 경고
notice = "warn"  # 보안 공지 경고

[licenses]
# 라이선스 정책
unlicensed = "deny"  # 라이선스 없는 크레이트 거부
copyleft = "warn"  # 카피레프트 라이선스 경고
default = "deny"  # 기본적으로 거부

[bans]
# 금지된 크레이트 관리
multiple-versions = "warn"  # 다중 버전 경고
wildcards = "allow"  # 와일드카드 버전 허용
```

Clippy 보안 규칙

clippy.toml의 보안 관련 설정:

```
# 금지된 메서드
disallowed-methods = [
"core::result::Result::unwrap", # unwrap() 사용 금지
"core::option::Option::unwrap", # unwrap() 사용 금지
]

# 금지된 타입
disallowed-types = [
"frame_support::storage::types::ValueQuery", # 안전하지 않은 쿼리 타입
]

# 복잡도 임계값
type-complexity-threshold = 5000
```

의존성 보안 검사

CI/CD 파이프라인의 자동 의존성 검토:

```yaml
dependency-review:
uses: actions/dependency-review-action@v3
with:
# 알려진 취약점 예외 처리
allow-ghsas: GHSA-pfrx-2q88-qq97, GHSA-w5p7-h5w8-2hfq, ...
```

허용된 GHSA 예외들:

- `GHSA-pfrx-2q88-qq97` : Docusaurus 빌드 관련
- `GHSA-969w-q74q-9j8v` : Substrate 전이 의존성
- `GHSA-fjx5-qpf4-xjf2` : ibc-proto-rs 전이 의존성

7.3.4 보안 취약점 관리

보안 정책 (SECURITY.md)

취약점 신고 프로세스:

```
# 보안 취약점 신고 요구사항

1. 취약점 설명

2. 플랫폼 정보 (OS, Rust 버전)

3. 재현 가능한 코드 샘플

4. 취약점 유형, 심각도, 영향도

5. 크레딧을 위한 이름 (선택사항)


# 신고 채널

- 이메일: security@composable.finance
```

책임감 있는 공개 정책:

- 2주간의 비공개 기간 또는 수정 완료까지
- 프로덕션 시스템 공격 금지
- PICA 토큰 보상 제공

취약점 심각도 분류

CVSS 기반 심각도 임계값:

```
# deny.toml에서 정의
severity-threshold = {
None = "0.0",
Low = "0.1 - 3.9",
Medium = "4.0 - 6.9",
High = "7.0 - 8.9",
Critical = "9.0 - 10.0"
}
```

7.3.5 의존성 관리 및 라이선스 컴플라이언스

허용된 의존성 소스

sources 섹션 설정:

```
[sources]
unknown-registry = "warn"  # 알 수 없는 레지스트리 경고
unknown-git = "warn"  # 알 수 없는 Git 저장소 경고

# 허용된 레지스트리
allow-registry = ["https://github.com/rust-lang/crates.io-index"]
```

```
# 허용된 Git 저장소
allow-git = []
```

라이선스 컴플라이언스

라이선스 정책 관리:

```
[licenses]
# 라이선스 검증 임계값
confidence-threshold = 0.8

# 크레이트별 예외 처리
exceptions = [
# 특정 크레이트에 대한 라이선스 예외
]

# 라이선스 명확화
[[licenses.clarify]]
name = "ring"
expression = "MIT AND ISC AND OpenSSL"
```

중복 의존성 관리

다중 버전 의존성 제어:

```
[bans]

# 중복 버전 허용 목록

skip = [

{ name = "wasi", version = "=0.10.0+wasi-snapshot-preview1" },

{ name = "smallvec", version = "=0.6.14" },

{ name = "sha2", version = "=0.9.9" },

# Substrate 자체가 여러 버전에 의존하므로 허용

]
```

7.3.6 코드 품질 보장 메커니즘

정적 분석 도구

Clippy 린팅 규칙:

```
#![cfg_attr(

not(test),

warn(

clippy::disallowed_methods, // 금지된 메서드 사용 경고

clippy::disallowed_types, // 금지된 타입 사용 경고

clippy::todo, // TODO 주석 경고

clippy::unwrap_used, // unwrap() 사용 경고

clippy::panic // panic! 사용 경고

)
```

```
)]
```

코드 포맷팅 표준

rustfmt.toml 설정을 통한 일관된 코드 스타일:

```
# 코드 포맷팅 규칙

max_width = 100

hard_tabs = true

tab_spaces = 4

newline_style = "Unix"
```

테스트 커버리지 요구사항

테스트 품질 보장:

```
// 각 팰릿의 테스트 요구사항

#[cfg(test)]

mod tests {

// 기능 테스트

// 에러 케이스 테스트

// 경계값 테스트

// 상태 일관성 테스트

}
```

7.3.7 보안 모니터링 및 대응

지속적 보안 모니터링

자동화된 보안 검사:

1. **의존성 취약점 스캔**: 매 PR마다 실행
2. **라이선스 컴플라이언스**: 자동 검증
3. **코드 품질 검사**: Clippy 및 포맷팅 검사
4. **보안 정책 준수**: deny.toml 규칙 적용

인시던트 대응 프로세스

보안 인시던트 처리 절차:

1. **즉시 대응**: 심각한 취약점 발견시 즉시 패치
2. **영향 분석**: 취약점의 범위 및 영향도 평가
3. **수정 배포**: 보안 패치 우선 배포
4. **사후 검토**: 인시던트 원인 분석 및 개선

보안 교육 및 인식 제고

개발팀 보안 교육:

1. **보안 코딩 가이드라인**: 안전한 코딩 관행 교육
2. **취약점 사례 연구**: 실제 취약점 사례 분석
3. **보안 도구 활용**: 보안 검사 도구 사용법 교육
4. **정기 보안 리뷰**: 주기적인 보안 상태 점검

7.3.8 규제 준수 및 감사 추적

감사 추적 관리

코드 변경 추적:

```
# REFERENCES 파일을 통한 감사 기준점 관리

f3a08dfbf6467047f213fa84e4c6a24f32d1f700  # 감사 기준 커밋
```

규제 준수 요구사항

컴플라이언스 체크리스트:

1. **라이선스 준수**: 모든 의존성의 라이선스 검증
2. **보안 표준**: 업계 보안 표준 준수
3. **감사 요구사항**: 정기적인 외부 보안 감사
4. **문서화**: 모든 보안 절차의 문서화

지속적 개선

보안 프로세스 개선:

1. **정기 검토**: 보안 정책 및 절차의 정기적 검토
2. **도구 업데이트**: 보안 도구 및 규칙의 지속적 업데이트
3. **피드백 반영**: 감사 결과 및 피드백의 적극적 반영
4. **베스트 프랙티스**: 업계 최신 보안 관행 도입

이러한 포괄적인 보안 감사 및 코드 리뷰 프로세스를 통해 Composable Finance 프로젝트는 높은 수준의 보안성과 코드 품질을 유지하며, 복잡한 DeFi 및 크로스체인 인프라의 안전성을 보장합니다.